ACTIVIDAD EVALUATIVA

EJE 3 - Entrega 3: Módulo de votaciones, validaciones y API GET

Sistema Web de Votaciones – Festival de la Leyenda Vallenata 2027

EQUIPO VOTACIONES FLV
SEBASTIÁN BAUTISTA MARTÍNEZ
MARÍA VIRGINIA LABARCA BRICEÑO
JUAN MEJÍA MAESTRE

PROFESOR

DEIVYS MORALES URIBE

DESARROLLO WEB - 4324 - 61
INGENIERÍA DE SISTEMAS
FUNDACIÓN UNIVERSITARIA DEL ÁREA ANDINA

NOVIEMBRE DE 2026

AREANDINA

Ficha de acceso y evidencias

Todos los enlaces son públicos y de solo lectura. El código se consulta sin iniciar sesión en GitHub; la API se ejecuta en local (`localhost`), por eso se demuestra en el video y se explica cómo ejecutarla desde el repositorio.

Tabla 1

Ficha de acceso de la Entrega 3

Elemento	Información
Equipo e integrantes	Equipo Votaciones FLV: Sebastián Bautista Martínez, María Virginia Labarca Briceño y Juan Mejía Maestre (tres integrantes, autorizados por el docente).
Repositorio	https://github.com/juandiegows/votaciones-festival-vallenato · rama principal `main`
README e instalación	https://github.com/juandiegows/votaciones-festival-vallenato/blob/main/README.md Sección «Entrega 3»: instalación, migraciones, datos de prueba y comandos
API GET	Ruta `GET /api/categorias/` · código: app/api/votaciones/votacion/views.py (`CategoriaPublicaViewSet`) y serializers.py (`CategoriaConOpcionesSerializer`) · minuto del video: [Pendiente: completar después de grabar]
Control de duplicados	app/api/votaciones/votacion/models.py (`UniqueConstraint` de `Voto`) y app/api/votaciones/votacion/servicios.py (`emitir_voto`, `except IntegrityError`) · minuto del video: [Pendiente: completar después de grabar]
Video	URL: [Pendiente: completar después de grabar] · duración total: [Pendiente: completar después de grabar] (máximo 8:00)
Evidencias por minuto	Ver la Tabla 2 (minutos reales de cada demostración).
Enlace adicional	Sitio desplegado (evidencia adicional): https://votaciones.juandiegows.com Guion del video: docs/entrega-3/guion-video.md Capturas de esta entrega: docs/entrega-3/capturas

Nota. Los textos resaltados se completan con el enlace y los minutos reales después de subir el video a YouTube como «No listado».

Tabla 2

Minutos del video por demostración

Momento orientativo	Demostración	Minuto real	Casos
0:00–0:45	Presentación del proyecto y del delegado	[minuto real]	—

Momento orientativo	Demostración	Minuto real	Casos
0:45–1:30	Inicio de sesión y categorías	[minuto real]	CP-04
1:30–2:30	Voto válido (selección, confirmación y registro)	[minuto real]	CP-01
2:30–3:30	Segundo voto en la misma categoría (bloqueo del duplicado)	[minuto real]	CP-02
3:30–4:15	Voto en otra categoría y check «Un solo voto en toda la categoría»	[minuto real]	CP-03, CP-05
4:15–5:00	Historial (Mis votos) y administración (Django Admin)	[minuto real]	CP-07, CP-08
5:00–6:00	API GET /api/categorias/	[minuto real]	CP-10
6:00–7:00	MySQL y código (restricción única, migraciones, CSRF)	[minuto real]	CP-09
7:00–8:00	Pruebas adicionales (API directa, pruebas automáticas) y cierre	[minuto real]	CP-05, CP-06, CP-09

Tabla 3

Rutas de los archivos principales

Qué	Archivo
Modelo `Voto` y restricción única; check `voto_unico` de la categoría	app/api/votaciones/votacion/models.py
Validaciones del voto y control de duplicados (incluida la concurrencia)	app/api/votaciones/votacion/servicios.py
Migraciones de la Entrega 3	0004_voto_unico_por_categoria_edicion.py 0005_voto_unico_configurable_por_categoria.py
Vista de voto (POST + CSRF), comprobante y «Mis votos»	app/api/portal/vistas/votos.py
Detalle de la votación (aviso «Ya votaste»)	app/api/portal/vistas/publico.py app/api/portal/templates/portal/votacion.html
API GET `/api/categorias/`	app/api/votaciones/votacion/views.py app/api/votaciones/votacion/serializers.py app/api/votaciones/votacion/selectores.py
Pruebas CP-01 a CP-10	app/api/portal/tests/test_entrega3.py
Dependencias y variables de entorno de ejemplo (sin secretos)	app/api/requirements.txt app/api/.env.example

AREANDINA

Tabla de Contenidos

Ficha de acceso y evidencias 2
Introducción y objetivo 5
Implementación técnica 6
  Modelos y relaciones 6
  Migraciones 7
  Vistas, rutas y formularios 8
  Validaciones del servidor 9
  Restricción de unicidad e integridad ante concurrencia 10
  API GET /api/categorias/ 10
Funcionalidades 11
Pruebas 20
Conclusiones y equipo 22
  Logros 22
  Dificultades 23
  Pendientes para la Entrega 4 23
  Aportes de los integrantes 23
Referencias 24

AREANDINA

Introducción y objetivo

Este documento corresponde a la Entrega 3 del proyecto de la asignatura Desarrollo Web. Continúa el proyecto Django + MySQL de la Entrega 2, que ya tenía la jerarquía **Edición → Categoría → Votación → Opción**, el sitio con vistas y plantillas de Django, la autenticación con sesión, el panel de administración propio y Django Admin. Sobre esa base se implementó el **módulo real de votaciones**: consultar categorías y opciones, registrar el voto con confirmación, validar en el servidor las restricciones, consultar la participación y exponer la API `GET /api/categorias/` con respuesta JSON.

Regla central. Cada usuario puede elegir una sola opción por categoría y edición. Puede votar en otras categorías, pero no emitir un segundo voto en la misma categoría, aunque esta contenga varias votaciones (subeventos). La regla se valida en Django y además la garantiza MySQL con una restricción de unicidad, incluso ante solicitudes simultáneas.

Objetivo. Integrar modelos, vistas, formularios, rutas, plantillas, persistencia y validaciones del servidor para construir un proceso de votación verificable: *Iniciar sesión → Elegir categoría → Seleccionar una opción → Confirmar → Validar en Django → Registrar en MySQL*.

Tabla 4

Funcionalidades obligatorias de la Entrega 3 y dónde se evidencian

Funcionalidad	Implementación	Evidencia
Categorías y opciones	Se leen de MySQL según la edición activa (`/2027/categorias/…`)	Figura 1 · CP-10
Autenticación	Sin sesión, votar lleva a `/login` y guarda la opción elegida	Figura 2 · CP-04
Registro de voto	Selección, ventana de confirmación y `POST` con CSRF; comprobante	Figuras 3 a 5 · CP-01

Funcionalidad	Implementación	Evidencia
Regla de unicidad	Validación en `emitir_voto` y `UniqueConstraint` en MySQL	Figuras 6, 7 y 19 · CP-02
Validaciones	Categoría habilitada, votación abierta, opción de la votación, sesión	Figuras 10, 11 y 17 · CP-05, CP-06
Participación y administración	«Mis votos» y Django Admin con filtros por edición y categoría	Figuras 12 y 13 · CP-07, CP-08
API GET obligatoria	`GET /api/categorias/`: categorías → votaciones → opciones	Figura 16 · CP-10
Integridad	`IntegrityError` → «Ya votaste»; prueba con dos hilos en MySQL	Figura 18 · CP-09

Implementación técnica

Modelos y relaciones

Se partió de los modelos de la Entrega 2. El modelo `Voto` ahora vincula **usuario, edición, categoría, votación y opción**. La edición y la categoría se copian de la votación al guardar (método `completar_ambito`), lo que permite que MySQL exija la regla con una restricción de unicidad. La categoría tiene el check `voto_unico` (marcado por defecto): marcado, la persona vota una sola vez en toda la categoría; desmarcado, una vez en cada votación de la categoría.

Tabla 5

Modelos que intervienen en el voto

Modelo (tabla)	Campos clave	Relaciones
Edicion (`edicion`)	año único, estado (activa/cerrada), pausa de votaciones	1 → N categorías
Categoria (`categoria`)	nombre y slug únicos por edición, activa, **voto_unico**	N → 1 edición; 1 → N votaciones
Votacion (`votacion`)	fechas de apertura y cierre, publicada, cerrada manualmente	N → 1 categoría; 1 → N opciones
Opcion (`opcion`)	nombre, activa, orden	N → 1 votación
Voto (`voto`)	codigo_comprobante único, fecha_hora, ip, **ambito_votacion**	N → 1 usuario, edición, categoría, votación y opción

Nota. `ambito_votacion` vale 0 cuando el voto cuenta para toda la categoría y el ID de la votación cuando la categoría no es de voto único.

```
class Voto(models.Model):
    usuario = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.PROTECT,
related_name="votos")
    edicion = models.ForeignKey(Edicion, on_delete=models.PROTECT, related_name="votos")
    categoria = models.ForeignKey(Categoria, on_delete=models.PROTECT,
related_name="votos")
    votacion = models.ForeignKey(Votacion, on_delete=models.PROTECT, related_name="votos")
    ambito_votacion = models.PositiveIntegerField(default=0, editable=False)
    opcion = models.ForeignKey(Opcion, on_delete=models.PROTECT, related_name="votos")
    ...
    class Meta:
        constraints = [
            # RN-04: un voto por usuario, categoría y edición (o por votación si la
categoría no es de voto único)
            models.UniqueConstraint(
                fields=["usuario", "categoria", "edicion", "ambito_votacion"],
                name="voto_unico_usuario_categoria_edicion",
            ),
        ]
```

Nota. Fragmento de `app/api/votaciones/votacion/models.py`.

Migraciones

Las migraciones se aplicaron con `python manage.py migrate` en MySQL 8.4 (entorno local con Docker) y en MySQL 9.7.2 (producción, el 27 de octubre de 2026). La **0004** agrega edición y categoría al voto, completa los votos existentes, descarta los repetidos de los datos de demostración (conserva el más antiguo) y crea la restricción única. La **0005** agrega el check `voto_unico` a la categoría y la columna `ambito_votacion` a la restricción. En producción depuró 198 votos repetidos, todos de cuentas ficticias de demostración (1043 → 845 votos). La Figura 19 muestra el índice único creado en MySQL.

Vistas, rutas y formularios

Tabla 6

Rutas del módulo de votaciones

Ruta	Método	Vista	Qué hace
`/{año}/categorias/{categoría}`	GET	`publico.categoria`	Votaciones de la categoría con la insignia «Ya votaste»
`/{año}/categorias/{categoría}/{votación}`	GET	`publico.votacion`	Opciones; aviso si ya votó en la categoría
`…/{votación}/votar`	POST	`votos.votar`	`@require_POST` + CSRF; exige sesión; llama a `emitir_voto`
`…/{votación}/comprobante`	GET	`votos.comprobante`	Código del comprobante y opción elegida
`/mis-votos`	GET	`votos.mis_votos`	Historial de participación del usuario
`/panel/categorias/{id}/editar`	GET/POST	`panel.categorias`	Check «Un solo voto en toda la categoría»
`/django-admin/votaciones/voto/`	GET	Django Admin	Consulta administrativa de votos (solo lectura)
`/api/categorias/`	GET	`CategoriaPublicaViewSet`	JSON con categorías, votaciones y opciones
`/api/votaciones/{id}/votar/`	POST	`VotacionPublicaViewSet.votar`	Mismo servicio `emitir_voto` (token)

El formulario de voto usa `{% csrf_token %}` y la vista solo acepta `POST`. Antes de enviar, el navegador muestra una ventana de confirmación (Figura 4). Las reglas no viven

en la plantilla ni en el botón: están en `servicios.emitir_voto`, que usan por igual el sitio Django y la API.

Validaciones del servidor

Tabla 7

Validaciones de emitir_voto

Validación	Respuesta	Código
Sesión iniciada (sitio) o token (API)	Redirige a `/login` · `401`	—
Correo confirmado y documento (si se exige)	Mensaje en la página · `403`	`correo_sin_confirmar`, `documento_requerido`
Categoría activa de la edición en curso	«La categoría no está habilitada para votar.» · `409`	`categoria_no_habilitada`
Votación abierta y sin pausa	«La votación no está abierta.» · `409`	`votacion_no_abierta`, `votaciones_pausadas`
La opción pertenece a la votación y está activa	«La opción no pertenece a esta votación.» · `400`	`opcion_invalida`
Sin voto previo en la categoría (o en la votación)	«Ya votaste en la categoría…» · `409`	`voto_duplicado_categoria`
Restricción única de MySQL (concurrencia)	`IntegrityError` → «Ya votaste…» · `409`	`voto_duplicado_categoria`

```
    # RN-04: una sola opción por categoría y edición (o por votación, si la categoría no es de voto único)
    if voto_que_bloquea(usuario, votacion):
        raise voto_duplicado(categoria)


    try:
        # Punto de guardado: si otra solicitud simultánea ganó, MySQL rechaza esta por la restricción única
        with transaction.atomic():
            return Voto.objects.create(usuario=usuario, edicion_id=categoria.edicion_id, categoria=categoria,
                                       votacion=votacion, opcion=opcion,

codigo_comprobante=generar_codigo_comprobante(votacion), ip=ip)
    except IntegrityError:
        # InnoDB hace esperar este INSERT hasta que la otra transacción confirme y luego lo rechaza por duplicado
        raise voto_duplicado(categoria) from None
```

Nota. Fragmento de `app/api/votaciones/votacion/servicios.py` (función `emitir_voto`).

Restricción de unicidad e integridad ante concurrencia

La validación previa da un mensaje claro, pero no basta sola: si dos solicitudes llegan al mismo tiempo, ambas podrían pasarla. Por eso la última barrera es MySQL. La restricción `voto_unico_usuario_categoria_edicion` hace que el segundo `INSERT` espere a que el primero confirme y luego falle por duplicado; Django captura el `IntegrityError` dentro de un punto de guardado y responde «Ya votaste». La prueba CP-09 lo comprueba con dos hilos reales contra MySQL (Figura 18).

API GET /api/categorias/

Se implementó con Django REST Framework. Devuelve JSON válido con las categorías activas de la edición (por defecto la activa; también `?anio=2027` o `?edicion={id}`), sus votaciones publicadas y las opciones activas de cada una. Las consultas se precargan con `Prefetch`, así la respuesta sale de MySQL en tres consultas aunque haya muchas categorías.

```
class CategoriaPublicaViewSet(viewsets.ReadOnlyModelViewSet):
    """GET /api/categorias/ (Entrega 3): JSON de MySQL con las categorías, sus votaciones y
sus opciones."""
    permission_classes = [AllowAny]
    serializer_class = CategoriaConOpcionesSerializer


class CategoriaConOpcionesSerializer(serializers.ModelSerializer):
    votaciones = VotacionConOpcionesSerializer(many=True, read_only=True,
source="votaciones_publicas")
    class Meta:
        model = Categoria
        fields = ["id", "edicion", "edicion_anio", "edicion_nombre", "nombre", "slug",
"descripcion",
                  "icono", "orden", "activa", "voto_unico", "votaciones"]
```

Nota. Fragmentos de `app/api/votaciones/votacion/views.py` y `serializers.py`.

Peticiones directas a la API. El botón deshabilitado es solo una ayuda visual: si alguien llama la API saltándose la interfaz, el servidor aplica las mismas reglas (Figura 17). La API exige token para votar, limita la frecuencia (`throttle` de 30 votos por minuto) y

solo admite peticiones del navegador desde los orígenes permitidos por CORS. Una API no puede distinguir con certeza si la petición viene del frontend o de otro cliente, por eso la seguridad está en las validaciones del servidor y en MySQL.

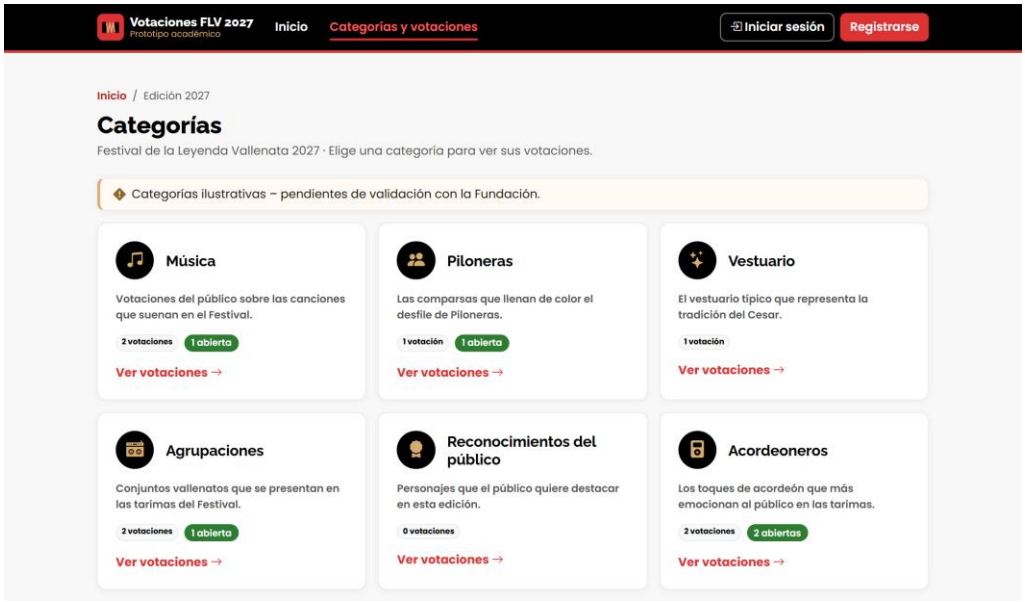
Secretos. El repositorio no incluye `.env` real ni credenciales; solo `app/api/.env.example` con valores de ejemplo. Las contraseñas de las cuentas de demostración se toman de variables de entorno en un servidor público.

Funcionalidades

Capturas reales tomadas el 27 de octubre de 2026 en el entorno local (Django + MySQL 8.4; producción usa MySQL 9.7.2 con el mismo código y las mismas migraciones) con los datos ficticios de `python manage.py cargar_demo --reiniciar`. Ninguna captura muestra contraseñas ni tokens.

Figura 1

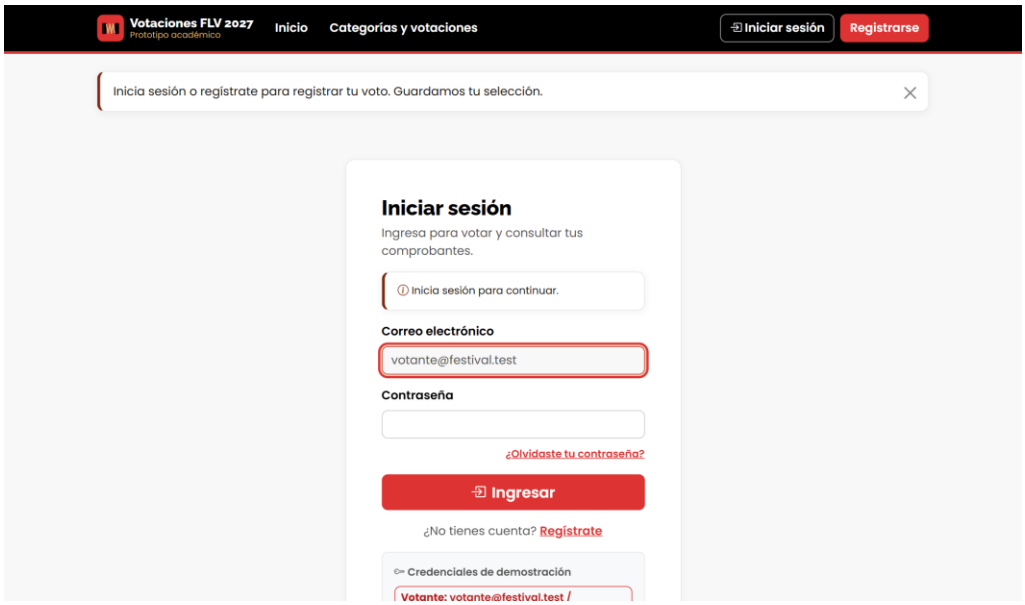
Categorías de la edición 2027 leídas de MySQL



Nota. Sitio Django sin sesión iniciada.

Figura 2

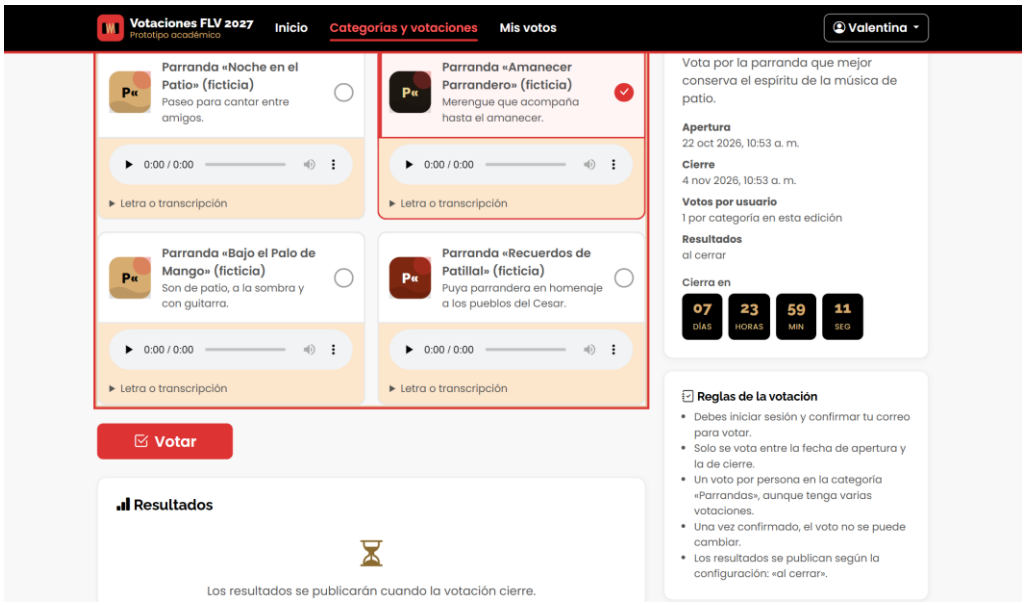
Sin sesión, votar lleva al inicio de sesión (CP-04)



Nota. La opción elegida se guarda y vuelve marcada al iniciar sesión.

Figura 3

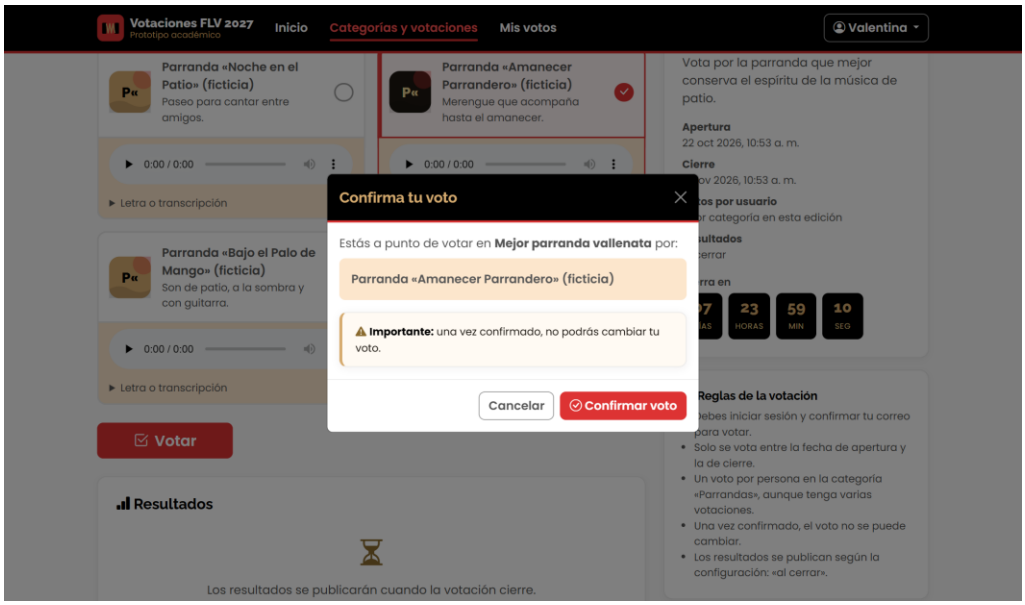
Opción seleccionada después de iniciar sesión



Nota. Votación «Mejor parranda vallenata» (categoría Parrandas, voto único).

Figura 4

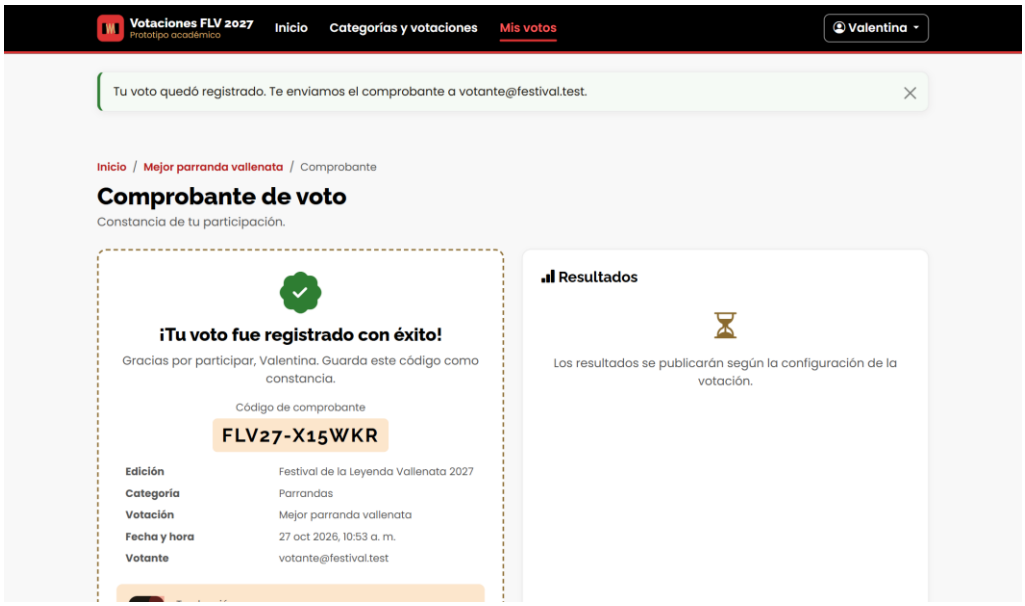
Confirmación del voto antes de registrarlo (CP-01)



Nota. Ventana de confirmación; el voto se envía por POST con el token CSRF.

Figura 5

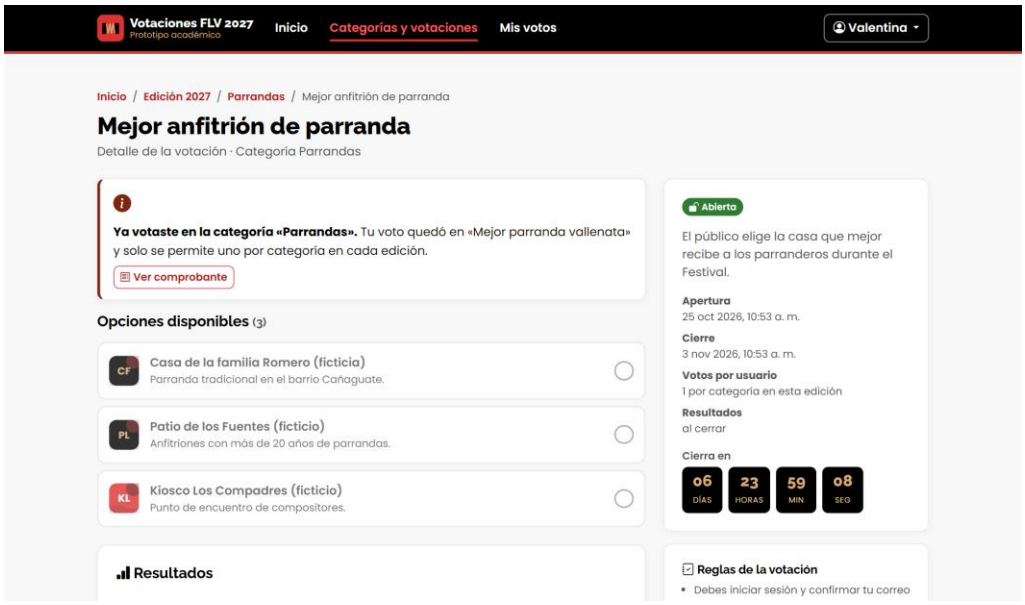
Comprobante del voto registrado en MySQL (CP-01)



Nota. Código FLV27-… generado al guardar el voto.

Figura 6

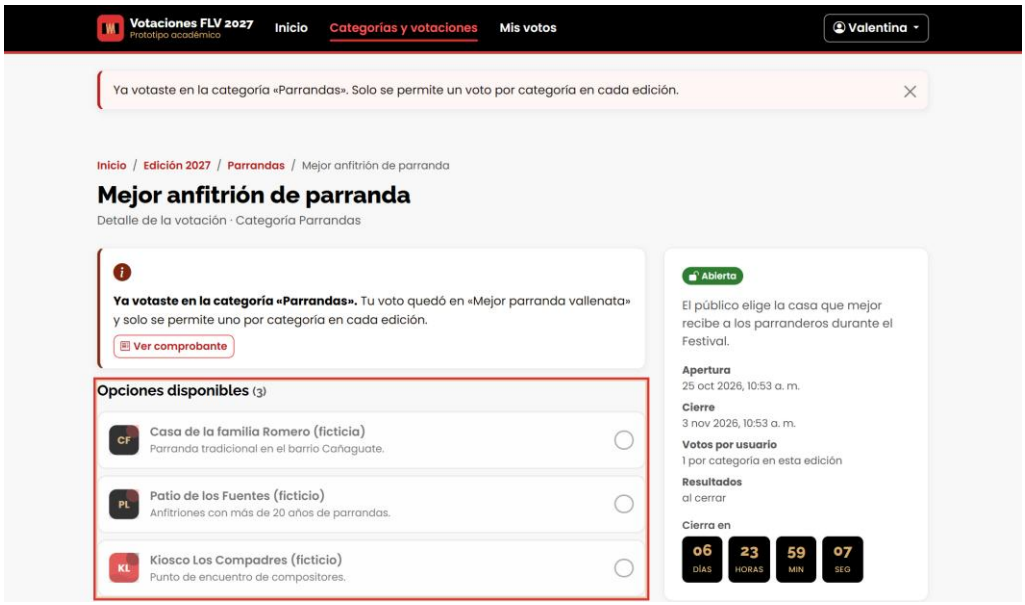
Otro subevento de la misma categoría: «Ya votaste en la categoría» (CP-02)



Nota. Votación «Mejor anfitrión de parranda»; no se muestra el botón Votar.

Figura 7

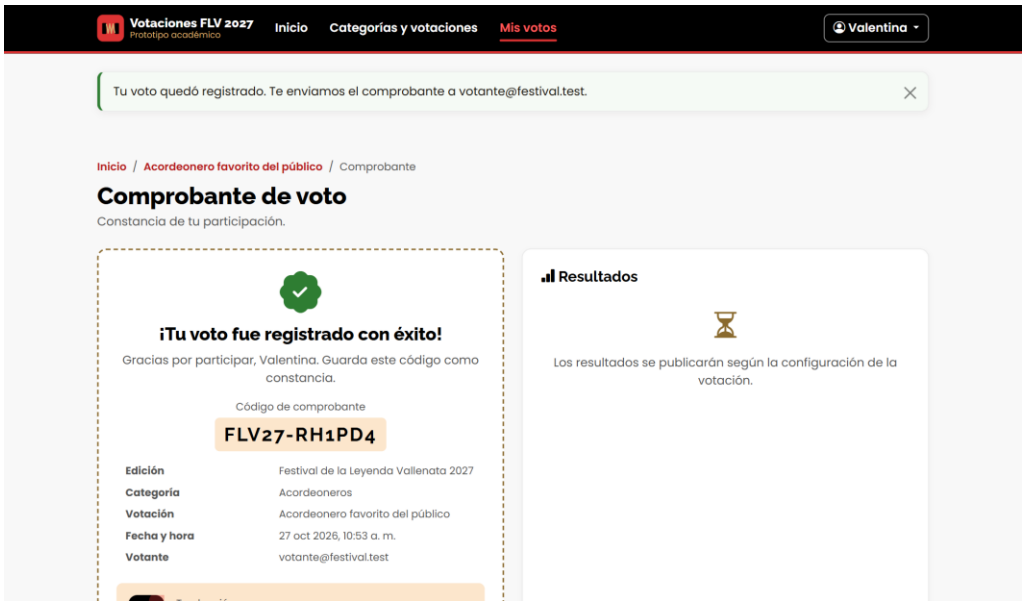
Django rechaza el segundo voto enviado desde una pestaña abierta antes de votar (CP-02)



Nota. La pestaña tenía el formulario; el servidor validó y no guardó el voto.

Figura 8

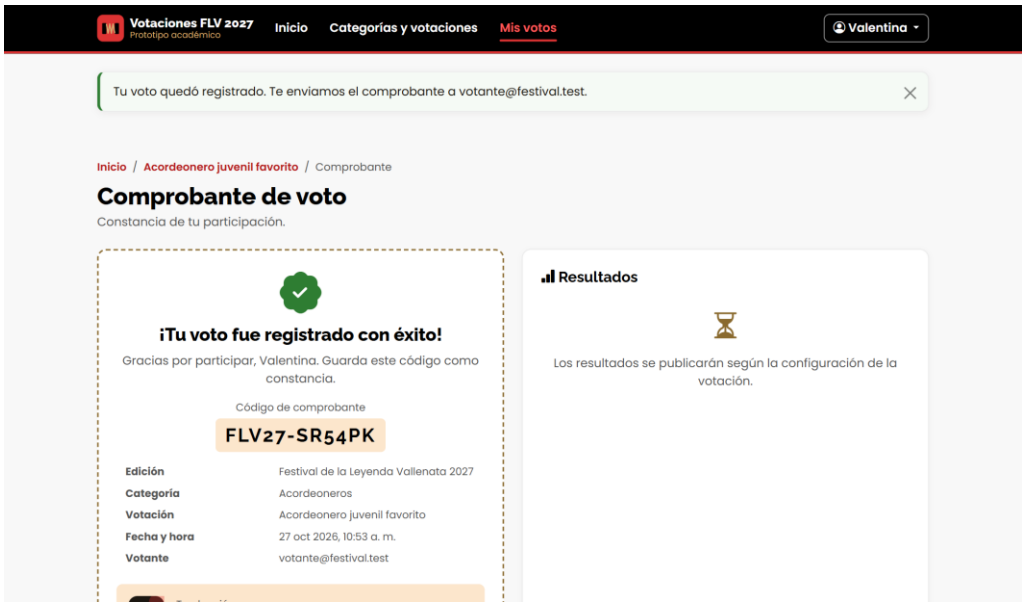
Voto permitido en otra categoría (CP-03)



Nota. Categoría Acordeoneros.

Figura 9

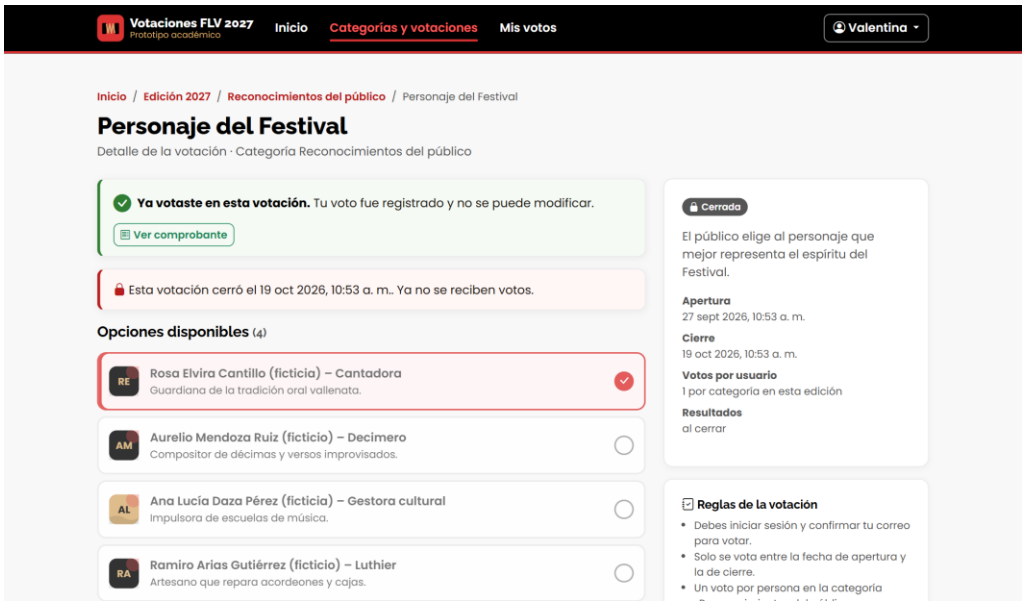
Categoría con el check desmarcado: se vota en cada una de sus votaciones



Nota. Acordeoneros tiene desmarcado «Un solo voto en toda la categoría».

Figura 10

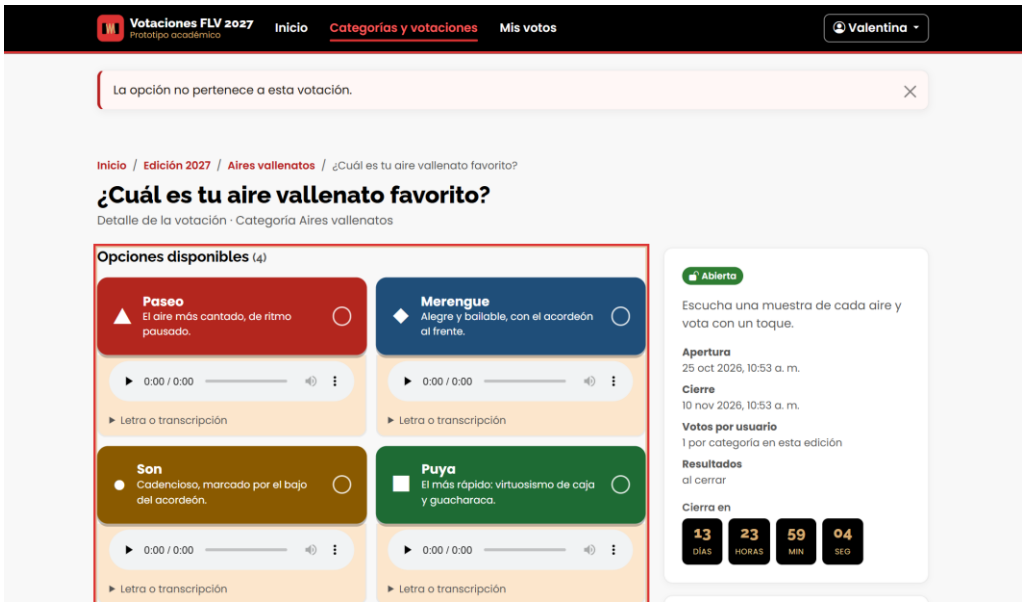
Votación cerrada: no se puede votar (CP-05)



Nota. «Personaje del Festival»; el servidor también lo rechaza (Figura 17).

Figura 11

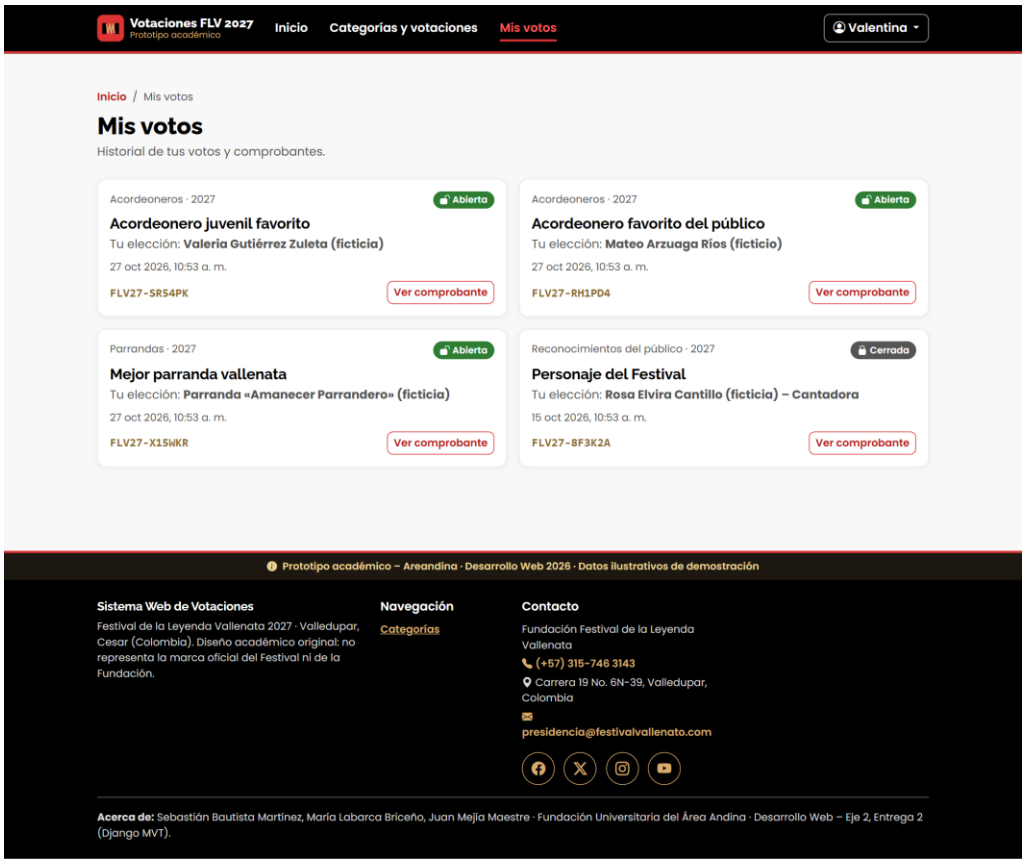
Se fuerza una opción de otra categoría y Django la rechaza (CP-06)



Nota. Se cambió el valor del formulario por una opción de «Personaje del Festival».

Figura 12

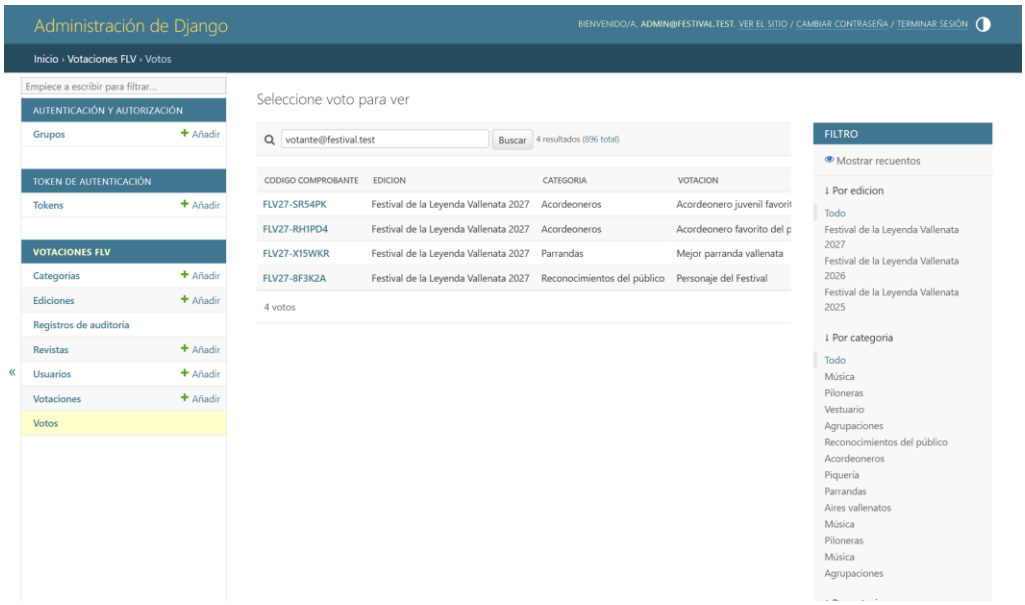
«Mis votos»: categorías votadas por el usuario (CP-07)



Nota. Coinciden con los registros de MySQL de la Figura 19.

Figura 13

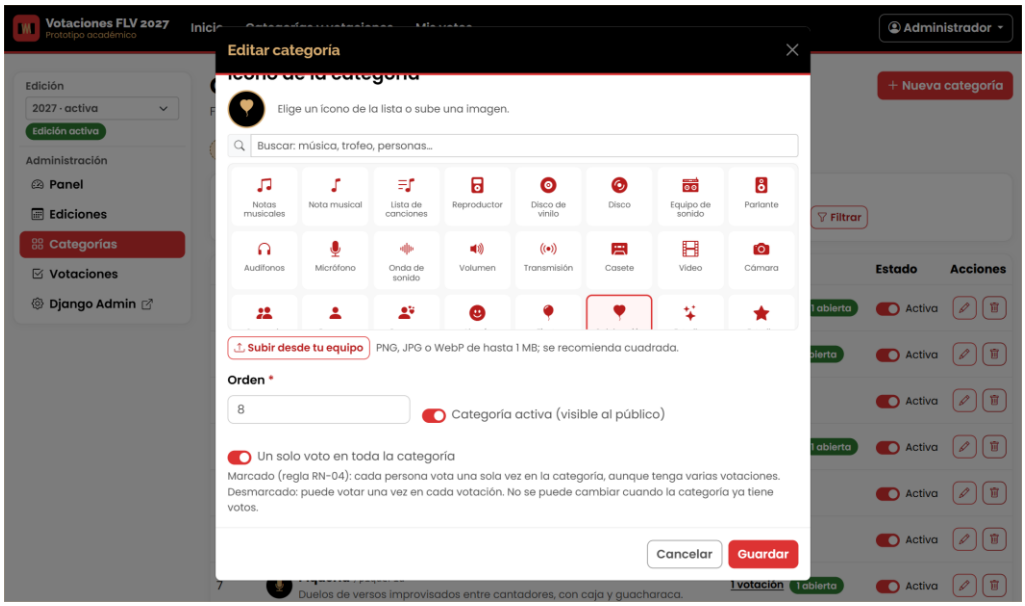
Consulta administrativa de votos en Django Admin (CP-08)



Nota. Búsqueda por el correo del votante; filtros por edición, categoría y votación.

Figura 14

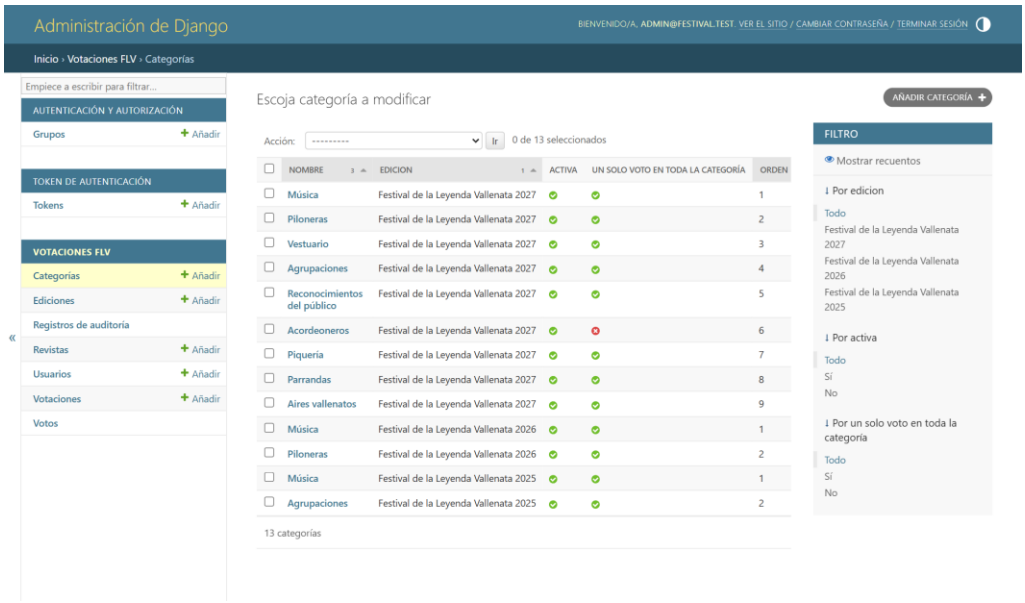
Check «Un solo voto en toda la categoría» en el panel



Nota. Categoría Parrandas. Con votos registrados el cambio se bloquea.

Figura 15

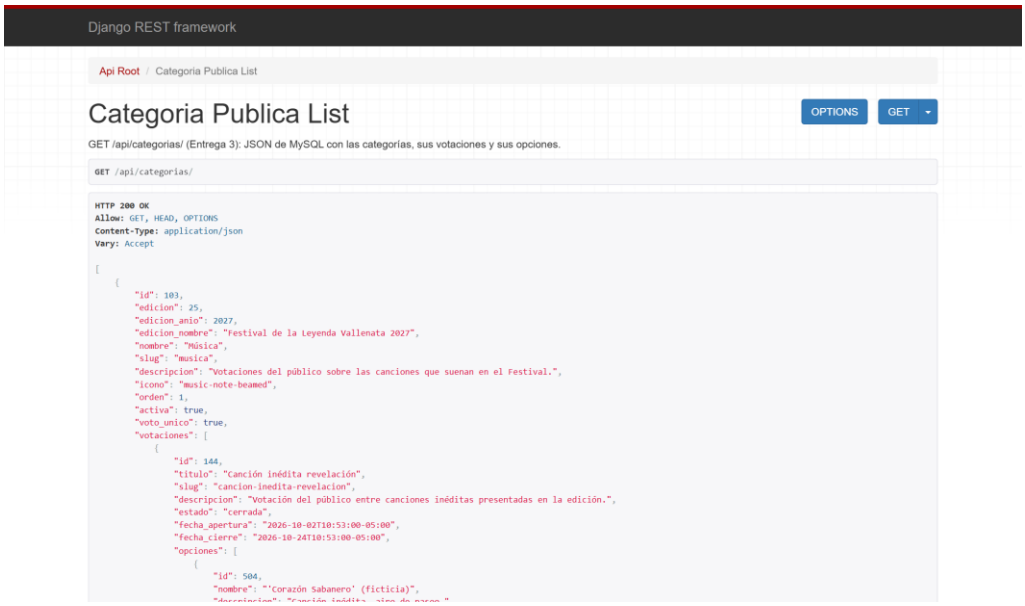
Modo de voto de cada categoría en Django Admin



Nota. Columna «Un solo voto en toda la categoría».

Figura 16

GET /api/categorias/: JSON con categorías, votaciones y opciones (CP-10)



Nota. Vista navegable de Django REST Framework; también responde JSON puro (Content-Type application/json).

Pruebas

Las diez pruebas obligatorias se automatizaron en `app/api/portal/tests/test_entrega3.py` y se ejecutaron contra MySQL 8.4 (Figura 18). Además se demostraron en la interfaz con las capturas indicadas. Toda la suite del proyecto (270 pruebas) también pasa.

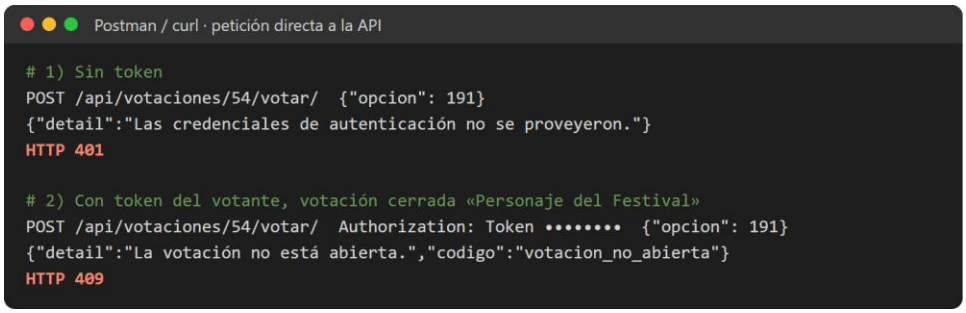
Tabla 8

Pruebas funcionales CP-01 a CP-10

Caso	Procedimiento	Resultado esperado	Resultado obtenido	Estado	Evidencia / minuto
CP-01	Votante con sesión elige una opción de «Mejor parranda vallenata», confirma	Se registra un voto en MySQL	Voto con comprobante FLV27-X15WKR	Aprobado	Fig. 4, 5, 19 · [minuto real]
CP-02	Intenta votar otra vez en Parrandas: otra opción y el subevento «Mejor anfitrión»	Se rechaza el segundo voto	«Ya votaste en la categoría «Parrandas»»; sigue 1 voto	Aprobado	Fig. 6, 7 · [minuto real]
CP-03	Vota en Acordeoneros	Se permite el voto	Voto registrado (FLV27-RH1PD4)	Aprobado	Fig. 8 · [minuto real]
CP-04	Visitante sin sesión pulsa Votar	Se exige autenticación	Redirige a /login?next=… con la opción guardada	Aprobado	Fig. 2 · [minuto real]
CP-05	Vota en «Personaje del Festival» (cerrada) desde el sitio y por la API	Se rechaza la operación	Botón deshabilitado; API 409 votacion_no_abierta	Aprobado	Fig. 10, 17 · [minuto real]
CP-06	Envía una opción que pertenece a otra categoría	Se rechaza la operación	«La opción no pertenece a esta votación.»	Aprobado	Fig. 11 · [minuto real]
CP-07	Consulta «Mis votos»	Se muestran registros correctos	Aparecen sus 4 votos con opción y comprobante	Aprobado	Fig. 12 · [minuto real]
CP-08	Administrador consulta los votos en Django Admin	Los datos coinciden con MySQL	Mismos comprobantes que la consulta SQL	Aprobado	Fig. 13, 19 · [minuto real]
CP-09	Dos hilos envían a la vez votos a dos subeventos de la misma categoría	Solo queda un voto válido	1 voto guardado; la otra solicitud recibe «Ya votaste»	Aprobado	Fig. 18, 19 · [minuto real]
CP-10	Consulta GET /api/categorias/	JSON válido con categorías y opciones reales	200 application/json con votaciones y opciones	Aprobado	Fig. 16 · [minuto real]

Figura 17

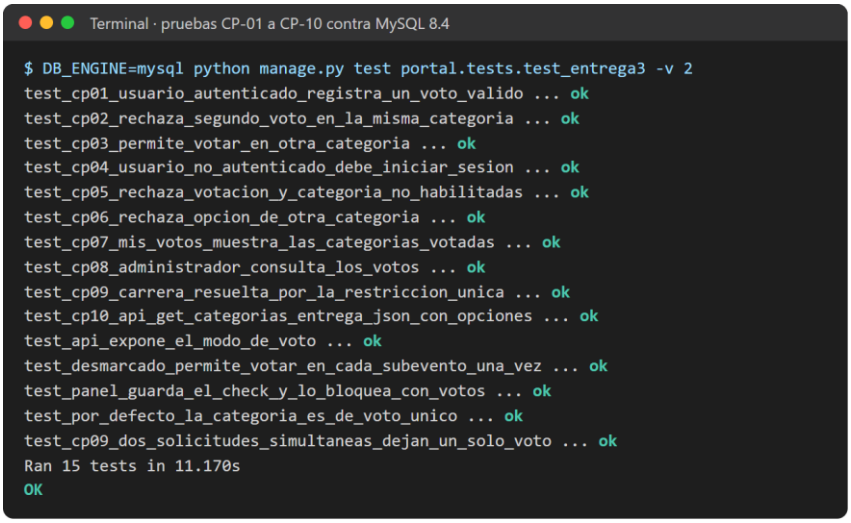
Peticiones directas a la API que se saltan el botón (CP-05)



Nota. Sin token: 401. Con el token del votante (oculto) en una votación cerrada: 409.

Figura 18

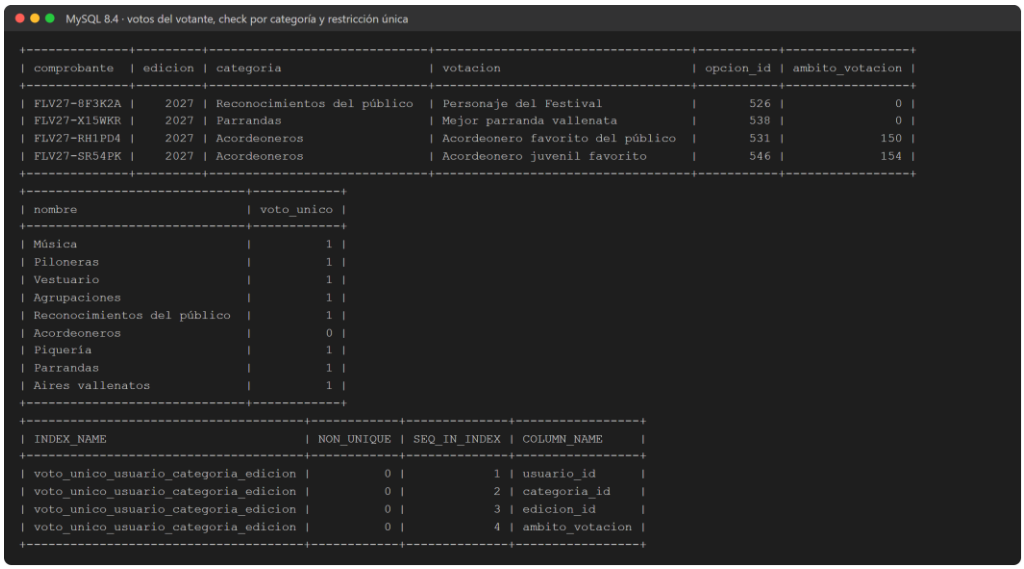
Pruebas automáticas de la Entrega 3 contra MySQL 8.4



Nota. Incluye CP-09 con dos solicitudes simultáneas reales (TransactionTestCase con hilos).

Figura 19

Votos del usuario en MySQL, check por categoría y restricción única



Nota. Consultas en MySQL 8.4 después de la demostración. ambito_votacion = 0 en categorías de voto único.

Conclusiones y equipo

Logros

- **Voto real y verificable.** El voto queda en MySQL con usuario, edición, categoría, votación y opción, y con un comprobante.
- **Regla central en dos capas.** Django valida y da el mensaje; MySQL garantiza la unicidad aunque lleguen solicitudes a la vez.
- **Una sola lógica para todas las entradas.** El sitio, la API y el panel usan el mismo servicio `emitir_voto`.
- **Configurable sin romper la regla.** El check por categoría permite votaciones independientes sin perder la integridad.

Dificultades

• **Datos previos.** El límite anterior era por votación; la migración 0004 tuvo que depurar votos repetidos de la demostración.

• **Concurrencia.** Con REPEATABLE READ una transacción no ve el voto que otra acaba de confirmar; se resolvió confiando en la restricción única.

Pendientes para la Entrega 4

• Pruebas integrales y de carga, correcciones y presentación final.

• Quitar del cliente React el ajuste antiguo de «votos por usuario», que ya no aplica.

• Validar con la Fundación las preguntas pendientes de la Entrega 1.

Aportes de los integrantes

Tabla 9

Aportes de cada integrante en la Entrega 3

Integrante	Aportes	Commits de la Entrega 3
Sebastián Bautista Martínez	Modelo `Voto`, migraciones 0004 y 0005, servicio `emitir_voto` con la restricción única y la API GET.	Exigir un voto por usuario, categoría y edición · Entregar votaciones y opciones en GET /api/categorias/ · Configurar por categoría el voto único con un check
María Virginia Labarca Briceño	Guion del video, documentación de la entrega y redacción del informe.	Agregar guion del video de la Entrega 3 · Actualizar el guion con el check de voto único y la API
Juan Mejía Maestre	Pantallas del voto («Ya votaste» por categoría), pruebas CP-01 a CP-10 y coordinación.	Mostrar «Ya votaste» en todas las votaciones de la categoría · Agregar pruebas CP-01 a CP-10 · Probar el check de voto único y el voto directo a la API

Nota. Los commits se pueden verificar en el historial del repositorio. La sustentación del video la realiza [Pendiente: completar después de grabar].

AREANDINA

Historial de commits: https://github.com/juandiegows/votaciones-festival-vallenato/commits/main

Referencias

Django Software Foundation. (2025). *Constraints reference* (versión 5.2). https://docs.djangoproject.com/en/5.2/ref/models/constraints/

Django Software Foundation. (2025). *Database transactions* (versión 5.2). https://docs.djangoproject.com/en/5.2/topics/db/transactions/

Django Software Foundation. (2025). *Cross Site Request Forgery protection* (versión 5.2). https://docs.djangoproject.com/en/5.2/ref/csrf/

Django Software Foundation. (2025). *Migrations* (versión 5.2). https://docs.djangoproject.com/en/5.2/topics/migrations/

Encode OSS. (2025). *Django REST framework: ViewSets*. https://www.django-rest-framework.org/api-guide/viewsets/

Encode OSS. (2025). *Django REST framework: Throttling*. https://www.django-rest-framework.org/api-guide/throttling/

Oracle Corporation. (2025). *MySQL 8.4 reference manual: InnoDB locking*. https://dev.mysql.com/doc/refman/8.4/en/innodb-locking.html